

Lab 6 DDPM

314551022 廖洺玄

Introduction

在這個 Lab 中，我們的主要任務是實作一個條件式 Denoising Diffusion Probabilistic Model (DDPM) 來根據多標籤條件生成合成 image。給定特定的標籤條件組合，例如 "red sphere"、"yellow cube" 和 "gray cylinder"...，我們的 DDPM 模型應該能夠生成包含這些物件的合成 image。此外，我們還需要將生成的 image 輸入預訓練的分類器 inference。在這個 Lab 我也有用到 LLM 來指導我如何實作 DDPM 的核心概念，包括 noise schedule、time embedding 和 sampling 方法的設計，以及幫我 debug 一些 code 和 improve 訓練策略。

Your implementation

Dataset and Data Loader

我使用了提供的 iclevr 資料集，其中包含了不同顏色和形狀的 3D 物件組合。我實作了一個客製化的 data loader 來處理多標籤條件：

```
class Iclevr_Dataset(Dataset):
    def __init__(self, mode='train'):
        self.object_dict = json.load(open('data/object.json'))
        self.data = json.load(open('data/train.json'))
        # convert dict to list of list
        self.data = [['data/iclevr/'+keys, self.data[keys]] for keys in self.data.keys()]
        def pil_to_tensor_no_numpy(im: Image.Image) -> torch.Tensor:
            im = im.convert('RGB')
            w, h = im.size
            # tobytes -> ByteStorage -> ByteTensor, avoid numpy path entirely
            byte_tensor = torch.ByteTensor(torch.ByteStorage.from_buffer(im.tobytes()))
            tensor_chw = byte_tensor.view(h, w, 3).permute(2, 0, 1).to(torch.float32) / 255.
            return tensor_chw

        self.transform = transforms.Compose([
            transforms.Resize((64, 64)),
            transforms.Lambda(pil_to_tensor_no_numpy),
            transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
        ])
    )
```

資料預處理方面，我將 `image transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))` 以符合預訓練分類器的要求，並且確保輸出解析度為 64x64。

Multi-label Condition Embedding

針對多標籤條件的 **embedding**，我設計了一個將物件標籤轉換為向量表示的方法。我使用 **one-hot encoding** 將每個物件的顏色和形狀組合編碼成固定維度的向量，然後透過 **linear layer** 將這些條件資訊融入到 DDPM 的 noise prediction 過程中。

Conditional DDPM Architecture

我的 DDPM 模型基於 U-Net 架構用以下設計，Backbone 用 U-Net 作為主要的 noise prediction 網路，在每個 ResNet block 中加入條件資訊，透過 **adaptive group normalization** 將標籤 embedding 注入到特徵中，使用 **sinusoidal position encoding** 來編碼時間步驟資訊。

```
)

# Define upward unet blocks
self.up_blocks = nn.ModuleList()
for i in range(n_downs, 0, -1):
    self.up_blocks.append(UNetUp(2**((i+1)*n_feat), 2**((i-1)*n_feat)))

# Define final convolutional layer
self.final_conv = nn.Sequential(
    nn.Conv2d(2*n_feat, n_feat, 3, 1, 1),
    nn.GroupNorm(8, n_feat),
    nn.GELU(),
    nn.Conv2d(n_feat, in_channels, 1, 1)
)

# Define time & context embedding blocks
self.timeembs = nn.ModuleList([EmbedFC(1, 2**i*n_feat) for i in range(n_downs, 0, -1)])
self.contextembs = nn.ModuleList([EmbedFC(n_cfeat, 2**i*n_feat) for i in range(n_downs, 0, -1)])

def forward(self, x, t, c):
    x = self.init_conv(x)
    downs = []
    for i, down_block in enumerate(self.down_blocks):
        if i == 0: downs.append(down_block(x))
```

Noise Schedule

我用了 **cosine scheduling** 而非 **linear scheduling**，因為我發現 **linear scheduling** 訓練效率很差。**Cosine scheduling** 讓模型能夠更有效地學習，避免在 **sampling** 上浪費。

```
def get_ddpm_noise_schedule_cosine(self, timesteps, beta1, beta2, device):
    """
    Returns DDPM noise schedule variables: a_t, b_t, ab_t
    b_t: \beta_t
    a_t: \alpha_t
    ab_t: \bar{\alpha}_t
    """
    import math
    # Use cosine schedule for beta_t
    t = torch.linspace(0, 1, timesteps+1, device=device)
    b_t = beta1 + 0.5 * (1 - torch.cos(t * math.pi)) * (beta2 - beta1)
    a_t = 1 - b_t
    ab_t = torch.cumprod(a_t, dim=0)
    return a_t, b_t, ab_t
```

Sampling Method

在 inference 階段，我實作了標準的 DDPM 和 DDIM sampling 過程，從 noise 開始 denoise

```
@torch.no_grad()
def sample_ddpm(self, n_samples, context=None, timesteps=None,
                 beta1=None, beta2=None, save_rate=20, inference_transform=lambda x: x, noise_schedule=None):
    """Returns the final denoised sample x0,
    intermediate samples xT, xT-1, ..., x1, and
    times tT, tT-1, ..., t1
    """
    if all([timesteps, beta1, beta2]):
        schedule = noise_schedule or "cosine"
        if schedule == "linear":
            a_t, b_t, ab_t = self.get_ddpm_noise_schedule(timesteps, beta1, beta2, self.device)
        else:
            a_t, b_t, ab_t = self.get_ddpm_noise_schedule_cosine(timesteps, beta1, beta2, self.device)
    else:
        timesteps, a_t, b_t, ab_t = self.get_ddpm_params_from_checkpoint(self.file_dir,
                                                                           self.checkpoint_name,
                                                                           self.device)

    self.nn_model.eval()
    samples = torch.randn(n_samples, self.nn_model.in_channels,
                           self.nn_model.height, self.nn_model.width,
                           device=self.device)
    intermediate_samples = [samples.detach().cpu()] # samples at T = timesteps
    t_steps = [timesteps] # keep record of time to use in animation generation
    for t in range(timesteps, 0, -1):
        print(f"Sampling timestep {t}", end="\r")
        if t % 50 == 0: print(f"Sampling timestep {t}")

        z = torch.randn_like(samples) if t > 1 else 0
        pred_noise = self.nn_model(samples,
                                     torch.tensor([t/timesteps], device=self.device)[: , None, None, None],
                                     context)
        samples = self.denoise_add_noise(samples, t, pred_noise, a_t, b_t, ab_t, z)

# fast sampling algorithm with context
@torch.no_grad()
def sample_ddim(n_sample, height, width, nn_model, timesteps, ab_t,
               device, context=None, n=20):
    #  $x_T \sim N(0, 1)$ , sample initial noise
    samples = torch.randn(n_sample, 3, height, width).to(device)

    # array to keep track of generated steps for plotting
    intermediate = []
    step_size = timesteps // n
    for i in range(timesteps, 0, -step_size):
        print(f'sampling timestep {i:3d}', end='\r')

        # reshape time tensor
        t = torch.tensor([i / timesteps])[: , None, None, None].to(device)

        eps = nn_model(samples, t, c=context) # predict noise  $\epsilon(x_t, t)$ 
        samples = denoise_ddim(samples, i, i - step_size, eps, ab_t)
        intermediate.append(samples.detach().cpu().numpy())

    intermediate = np.stack(intermediate)
    return samples, intermediate
```

Loss Function

我選擇使用 L2 (MSE) loss 作為主要的 training loss function，這是 DDPM 論文中推薦的做法。雖然這讓 model 在 learning shape feature（特別是圓形和圓柱形的重疊區域）時有些困難，但整體訓練穩定性較好。

```
print('=====Experiment_group={0}=====' .format(experiment_group))

for x, c in tqdm(dataloader, mininterval=2, desc=f"Epoch {epoch}"):
    x = x.to(self.device)
    c = c.to(self.device)

    # perturb data
    noise = torch.randn_like(x)
    t = torch.randint(1, timesteps + 1, (x.shape[0],)).to(self.device)
    x_pert = self.perturb_input(x, t, noise, ab_t)

    # predict noise
    pred_noise = self.nn_model(x_pert, t / timesteps, c=c)

    # obtain loss
    loss = torch.nn.functional.mse_loss(pred_noise, noise)

    # update params
    optim.zero_grad()
    loss.backward()
    optim.step()

    ave_loss += loss.item()/len(dataloader)
scheduler.step()
```

Training Implementation

我首先會根據隨機選擇的時間步驟 t 對原始 image 加入對應程度的 noise，然後將 noise image、時間 embedding 和條件資訊輸入模型預測 noise 並計算預測 noise 與真實 noise 之間的 MSE loss 接下來用 Adam optimizer 更新模型 parameters model weights.

```

def train(self, batch_size=64, n_epoch=32, lr=1e-3, timesteps=500, beta1=1e-4, beta2=0.02,
          checkpoint_save_dir=None, image_save_dir=None, noise_schedule="cosine"):
    """Trains model for given inputs"""
    self.nn_model.train()
    if noise_schedule == "linear":
        _, _, ab_t = self.get_ddpm_noise_schedule(timesteps, beta1, beta2, self.device)
    else:
        _, _, ab_t = self.get_ddpm_noise_schedule_cosine(timesteps, beta1, beta2, self.device)
    # prepare logging
    log_dir = os.path.join(self.file_dir, "logs")
    os.makedirs(log_dir, exist_ok=True)
    csv_path = os.path.join(log_dir, f"{self.dataset_name}_train_loss.csv")
    loss_history = []
    epoch_history = []
    if self.dataset_name == "iclevr":
        dataset = Iclevr_Dataset()
    else:
        dataset = self.instantiate_dataset(self.dataset_name,
                                           self.get_transforms(self.dataset_name), self.file_dir)
    dataloader = self.initialize_dataloader(dataset, batch_size, self.checkpoint_name, self.file_dir)
    optim = self.initialize_optimizer(self.nn_model, lr, self.checkpoint_name, self.file_dir, self.device)
    scheduler = self.initialize_scheduler(optim, self.checkpoint_name, self.file_dir, self.device)

    start_epoch = self.get_start_epoch(self.checkpoint_name, self.file_dir)
    for epoch in range(start_epoch, start_epoch + n_epoch):
        ave_loss = 0
        print(f"lr: {optim.param_groups[0]['lr']}")

        for x, c in tqdm(dataloader, mininterval=2, desc=f"Epoch {epoch}"):
            x = x.to(self.device)
            c = c.to(self.device)

            # perturb data
            noise = torch.randn_like(x)
            t = torch.randint(1, timesteps + 1, (x.shape[0],)).to(self.device)
            x_pert = self.perturb_input(x, t, noise, ab_t)

            # predict noise
            pred_noise = self.nn_model(x_pert, t / timesteps, c=c)

            # obtain loss
            loss = torch.nn.functional.mse_loss(pred_noise, noise)

            # update params
            optim.zero_grad()
            loss.backward()
            optim.step()

            ave_loss += loss.item() / len(dataloader)
        scheduler.step()
        print(f"Epoch: {epoch}, loss: {ave_loss}")
        # log to memory and csv
        loss_history.append(ave_loss)
        epoch_history.append(epoch)
        if not os.path.exists(csv_path):
            with open(csv_path, 'w') as f:
                f.write('epoch,loss\n')
        with open(csv_path, 'a') as f:
            f.write(f"{epoch},{ave_loss}\n")
        # save curve png
        try:
            import matplotlib.pyplot as plt
            plt.figure()

```

我使用 Adam optimizer，learning rate 設為 $2e-4$ ，並且發現 DDPM 相比 GAN 更容易訓練，在約 20 個 epoch 後就能產生正常的 image 結果。

Results and Discussion

Synthetic Image Grids

Test.json Results

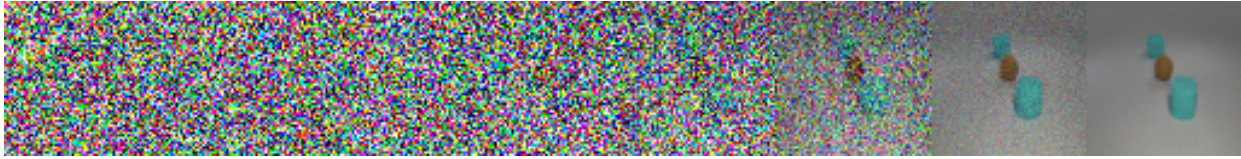


New_test.json Results



Denoising Process Visualization

根據要求標籤組合 ["red sphere", "cyan cylinder", "cyan cube"] 的去 noise 過程:



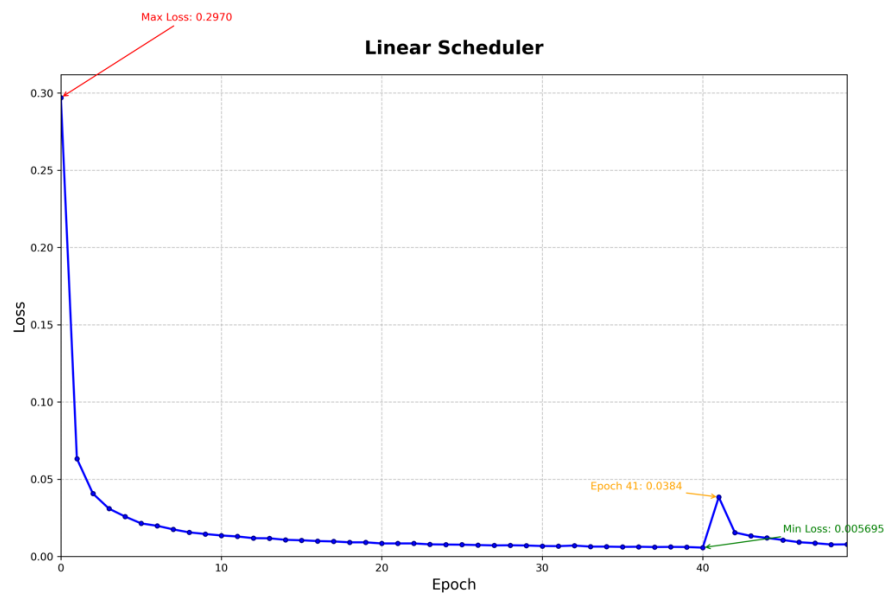
Performance Analysis

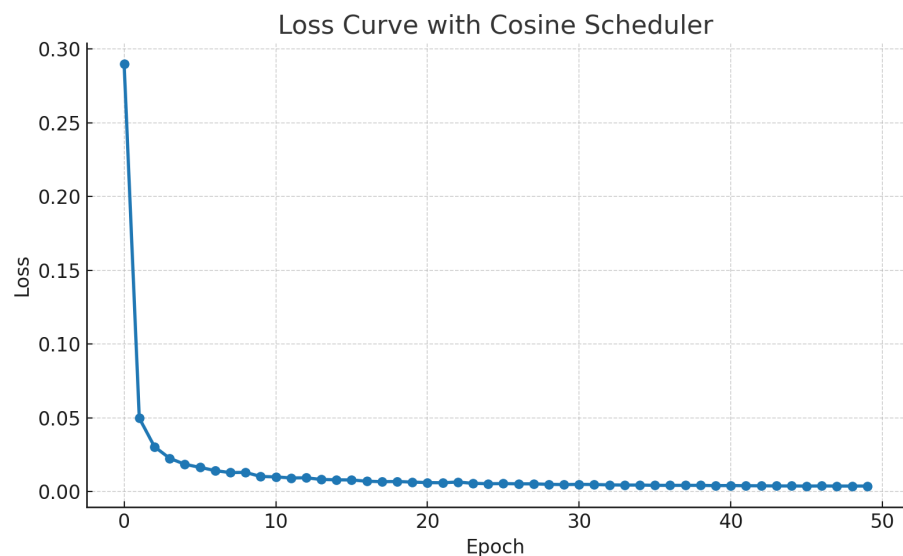
從實驗結果可以看出，我的 DDPM 模型在單一物件的條件生成上表現良好，但在處理多物件條件時偶爾會出現物件數量錯誤的問題。這主要是因為 **MSE loss** 的特性，它傾向於在像素層面計算損失，而非真正理解物件的語義資訊。

Extra Implementations and Experiments

Noise Schedule Comparison

我最初嘗試了 **linear noise schedule**，但發現前 400 個步驟基本上只產生純噪聲，訓練效率很低而且訓練到後面還會不穩定，改用 **cosine schedule** 後，模型能夠更好地利用每個訓練步驟，收斂速度明顯提升。





Training Straegie

我在訓練時是分成 4 個 part 去 train 因為我發現我訓練到後面雖然已經收斂的但是 loss 都下不去，所以我就用剛剛 train 好的 pre-train model 條一下 lr 然後繼續去 train



Experimental results

我用的 script 是 `python3 eval.py --run test`

Test.json Results

Classification Accuracy: 0.6071


```

Sampling timestep 300
Sampling timestep 250
Sampling timestep 200
Sampling timestep 150
Sampling timestep 100
Sampling timestep 50
/home/llm132/miniconda3/envs/lab6/lib/python3.10/site-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.
  warnings.warn(
/home/llm132/miniconda3/envs/lab6/lib/python3.10/site-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=None`.
  warnings.warn(msg)
Accuracy on test.json: 0.6071
(llm132) llm132@llm132-server:~/iii$

```

我用的 script 是 `python3 eval.py --run new_test`

New_test.json Results

Classification Accuracy: 0.631

```

Sampling timestep 200
Sampling timestep 150
Sampling timestep 100
Sampling timestep 50
Accuracy on new_test.json: 0.6310
(llm132) llm132@llm132-server:~/iii$ python3 eval.py
--device cuda:1

```

Reference

1. <https://adam-study-note.medium.com/diffusion-model-denoising-diffusion-probabilistic-models-ddpm-%E8%A9%B3%E7%B4%B0%E4%BB%8B%E7%B4%B9-5ce77b6b64d4>
2. <https://arxiv.org/abs/2006.11239>
3. https://blog.csdn.net/qq_40714949/article/details/126643111
4. <https://github.com/jonathan-liu-0204>